

24 - Sorting Algorithms

Acabamos de ver que si sabemos que una lista está ordenada, podemos explotar esa información para reducir significativamente el tiempo necesario para buscar en una lista. Esto significa que cuando se nos pide buscar en una lista, primero deberíamos ordenarla y luego realizar la búsqueda.

Sea $\theta(\text{sortComplexity}(L))$ una cota ajustada sobre la complejidad de ordenar una lista. Dado que sabemos que podemos buscar en una lista no ordenada en tiempo $\theta(\text{len}(L))$, la pregunta de si deberíamos primero ordenar y luego buscar se reduce a la pregunta: ¿es $\text{sortComplexity}(L) + \log(\text{len}(L))$ menor que $\text{len}(L)$? La respuesta, lamentablemente, es no. Es imposible ordenar una lista sin observar cada elemento de la lista al menos una vez, por lo que no es posible ordenar una lista en tiempo sublineal.

¿Significa esto que la búsqueda binaria carece de importancia práctica? Afortunadamente, no. Supongamos que esperamos buscar en la misma lista muchas veces. Podría tener mucho sentido pagar el costo adicional de ordenar la lista una vez, y luego amortizar el costo del ordenamiento a lo largo de muchas búsquedas. Si esperamos buscar en la lista k veces, la pregunta relevante se convierte en: ¿es $\text{sortComplexity}(L) + k \cdot \log(\text{len}(L))$ menor que $k \cdot \text{len}(L)$?

A medida que k se vuelve grande, el tiempo requerido para ordenar la lista se vuelve cada vez más irrelevante. ¡Qué grande necesita ser k depende de cuánto tiempo toma ordenar una lista! Si, por ejemplo, el ordenamiento fuera exponencial en el tamaño de la lista, k tendría que ser bastante grande.

Afortunadamente, el ordenamiento se puede hacer de manera bastante eficiente. Por ejemplo, la implementación estándar del ordenamiento en la mayoría de las implementaciones de Python se ejecuta en aproximadamente $O(n \cdot \log(n))$ tiempo, donde n es la longitud de la lista. En la práctica, rara vez necesitarás implementar tu propia función de ordenamiento. En la mayoría de los casos, lo correcto es usar el método incorporado `sort` de Python (`L.sort()` ordena la lista `L` in situ) o su función incorporada `sorted` (`sorted(L)` devuelve una nueva lista con los mismos elementos que `L`, pero no modifica `L`). Presentamos aquí algoritmos de ordenamiento principalmente para proporcionar algo

de práctica en el pensamiento sobre diseño de algoritmos y análisis de complejidad.

Comenzamos con un algoritmo simple pero ineficiente, **ordenamiento por selección**. El ordenamiento por selección funciona manteniendo el **invariante de bucle** de que, dado un particionado de la lista en un prefijo (`L[0:i]`) y un sufijo (`L[i+1:len(L)]`), el prefijo está ordenado y ningún elemento en el prefijo es mayor que el elemento más pequeño en el sufijo.

Usamos inducción para razonar sobre los invariantes de bucle.

- **Caso base:** Al inicio de la primera iteración, el prefijo está vacío, es decir, el sufijo es toda la lista. Por lo tanto, el invariante es (trivialmente) verdadero.
- **Paso inductivo:** En cada paso del algoritmo, movemos un elemento del sufijo al prefijo. Hacemos esto agregando un elemento mínimo del sufijo al final del prefijo. Debido a que el invariante se cumplía antes de mover el elemento, sabemos que después de agregar el elemento, el prefijo sigue ordenado. También sabemos que dado que removimos el elemento más pequeño del sufijo, ningún elemento en el prefijo es mayor que el elemento más pequeño en el sufijo.
- **Terminación:** Cuando se sale del bucle, el prefijo incluye toda la lista, y el sufijo está vacío. Por lo tanto, toda la lista ahora está ordenada en orden ascendente.

Ordenamiento por selección

```
1  def sel_sort(L):
2      """Asume que L es una lista de elementos que pueden ser
3          comparados usando >.
4          Ordena L en orden ascendente"""
5      suffix_start = 0
6      while suffix_start != len(L):
7          #examinar cada elemento en el sufijo
8          for i in range(suffix_start, len(L)):
9              if L[i] < L[suffix_start]:
10                 #intercambiar posición de elementos
11                 L[suffix_start], L[i] = L[i], L[suffix_start]
12             suffix_start += 1
```

Es difícil imaginar un algoritmo de ordenamiento más simple o más obviamente correcto. Desafortunadamente, es bastante ineficiente. La complejidad del bucle interno es $\theta(\text{len}(L))$. La complejidad del bucle externo también es $\theta(\text{len}(L))$. Por lo tanto, la complejidad de toda la función es $\theta(\text{len}(L)^2)$. Es decir, es cuadrática en la longitud de L.

Ordenamiento por mezcla (Merge Sort)

Afortunadamente, podemos hacerlo mucho mejor que tiempo cuadrático usando un **algoritmo divide y vencerás**. La idea básica es combinar soluciones de instancias más simples del problema original. En general, un algoritmo divide y vencerás se caracteriza por:

- Un tamaño umbral de entrada, por debajo del cual el problema no se subdivide
- El tamaño y número de sub-instancias en las que se divide una instancia
- El algoritmo usado para combinar sub-soluciones.

El umbral a veces se llama **caso base recursivo**. Para el elemento 2, es usual considerar la razón del tamaño del problema inicial al tamaño de la sub-instancia. En la mayoría de los ejemplos que hemos visto hasta ahora, la razón era 2.

Merge sort es un algoritmo prototípico de divide y vencerás. Fue inventado en 1945, por John von Neumann, y aún se usa ampliamente. Como muchos algoritmos divide y vencerás, se describe más fácilmente de forma recursiva:

1. Si la lista es de longitud 0 o 1, ya está ordenada.
2. Si la lista tiene más de un elemento, divide la lista en dos listas, y usa merge sort para ordenar cada una de ellas.
3. Mezcla los resultados.

La observación clave hecha por von Neumann es que dos listas ordenadas pueden mezclarse eficientemente en una sola lista ordenada. La idea es mirar el primer elemento de cada lista y mover el menor de los dos al final de la lista resultado. Cuando una de las listas está vacía, todo lo que queda es copiar los elementos restantes de la otra lista.

Considera, por ejemplo, mezclar las dos listas `L_1 = [1,5,12,18,19,20]` y `L_2 = [2,3,4,17]`:

Restante en L_1	Restante en L_2	Resultado
[1,5,12,18,19,20]	[2,3,4,17]	[]
[5,12,18,19,20]	[2,3,4,17]	[1]
[5,12,18,19,20]	[3,4,17]	[1,2]
[5,12,18,19,20]	[4,17]	[1,2,3]
[5,12,18,19,20]	[17]	[1,2,3,4]

Restante en L_1	Restante en L_2	Resultado
[12,18,19,20]	[17]	[1,2,3,4,5]
[18,19,20]	[17]	[1,2,3,4,5,12]
[18,19,20]	[]	[1,2,3,4,5,12,17]
[]	[]	[1,2,3,4,5,12,17,18,19,20]

¿Cuál es la complejidad del proceso de mezcla? Involucra dos operaciones de tiempo constante, comparar los valores de elementos y remover elementos de una lista a otra. El número de comparaciones es del orden $\theta(\text{len}(L))$, donde L es la más larga de las dos listas. El número de operaciones de copia es del orden $\theta(\text{len}(L_1) + \text{len}(L_2))$, porque cada elemento se copia exactamente una vez. (El tiempo para copiar un elemento depende del tamaño del elemento. Sin embargo, esto no afecta el orden del crecimiento del ordenamiento como función del número de elementos en la lista.) Por lo tanto, mezclar dos listas ordenadas es lineal en la longitud de las listas.

Implementación del algoritmo merge sort:

Merge sort

```

1  def merge(left, right, compare):
2      """Asume que left y right son listas ordenadas y
3          compare define un ordenamiento sobre los elementos.
4          Devuelve una nueva lista ordenada (por compare) que contiene los
5          mismos elementos que (left + right) contendría."""
6
7      result = []
8      i, j = 0, 0
9      while i < len(left) and j < len(right):
10         if compare(left[i], right[j]):
11             result.append(left[i])
12             i += 1
13         else:
14             result.append(right[j])
15             j += 1
16     while (i < len(left)):
17         result.append(left[i])
18         i += 1
19     while (j < len(right)):
20         result.append(right[j])
21         j += 1
22     return result
23
24 def merge_sort(L, compare = lambda x, y: x < y):
25     """Asume que L es una lista, compare define un ordenamiento
26         sobre los elementos de L
27         Devuelve una nueva lista ordenada con los mismos elementos que L"""
28     if len(L) < 2:

```

```

29         return L[:]
30     else:
31         middle = len(L)//2
32         left = merge_sort(L[:middle], compare)
33         right = merge_sort(L[middle:], compare)
34         return merge(left, right, compare)

```

Observa que hemos hecho del operador de comparación un parámetro de la función `merge_sort` y escrito una expresión lambda para proporcionar un valor por defecto. Así, por ejemplo, el código

```

1  L = [2,1,4,5,3]
2  print(merge_sort(L), merge_sort(L, lambda x, y: x > y))

```

Imprime

```

1  [1, 2, 3, 4, 5] [5, 4, 3, 2, 1]

```

Analicemos la complejidad de `merge_sort`. Ya sabemos que la complejidad temporal de `merge` es del orden $\theta(\text{len}(L))$. En cada nivel de recursión, el número total de elementos a ser mezclados es $\text{len}(L)$. Por lo tanto, la complejidad temporal de `merge_sort` es del orden $\theta(\text{len}(L))$ multiplicado por el número de niveles de recursión. Dado que `merge_sort` divide la lista a la mitad cada vez, sabemos que el número de niveles de recursión es del orden $\theta(\log(\text{len}(L)))$. Por lo tanto, la complejidad temporal de `merge_sort` es $\theta(n * \log(n))$, donde n es $\text{len}(L)$.

Esto es mucho mejor que $\theta(\text{len}(L)^2)$ del ordenamiento por selección. Por ejemplo, si L tiene 10,000 elementos, $\text{len}(L)^2$ es 100 millones pero $\text{len}(L) * \log_2(\text{len}(L))$ es aproximadamente 130,000.

Esta mejora en complejidad temporal viene con un costo. El ordenamiento por selección es un ejemplo de un algoritmo de **ordenamiento en el lugar**. Debido a que funciona intercambiando la posición de elementos dentro de la lista, usa solo una cantidad constante de almacenamiento extra (un elemento en nuestra implementación). En contraste, el algoritmo merge sort involucra hacer copias de la lista. Esto significa que su complejidad espacial es del orden $\theta(\text{len}(L))$. Esto puede ser un problema para listas grandes.

Supongamos que queremos ordenar una lista de nombres escritos como nombre seguido por apellido, por ejemplo, la lista `['Chris Terman', 'Tom Brady', 'Eric Grimson', 'Gisele Bundchen']`. El siguiente código define dos funciones de ordenamiento, y luego usa estas para ordenar una lista de dos maneras diferentes. Cada función usa el método `split` del tipo `str`.

Ordenando una lista de nombres

```
1  def last_name_first_name(name1, name2):
2      arg1 = name1.split(' ')
3      arg2 = name2.split(' ')
4      if arg1[1] != arg2[1]:
5          return arg1[1] < arg2[1]
6      else: #apellidos iguales, ordenar por nombre
7          return arg1[0] < arg2[0]
8
9  def first_name_last_name(name1, name2):
10     arg1 = name1.split(' ')
11     arg2 = name2.split(' ')
12     if arg1[0] != arg2[0]:
13         return arg1[0] < arg2[0]
14     else: #nombres iguales, ordenar por apellido
15         return arg1[1] < arg2[1]
16
17  L = ['Tom Brady', 'Eric Grimson', 'Gisele Bundchen']
18  newL = merge_sort(L, last_name_first_name)
19  print('Sorted by last name =', newL)
20  newL = merge_sort(L, first_name_last_name)
21  print('Sorted by first name =', newL)
```

Cuando el código se ejecuta, imprime

```
1  Sorted by last name = ['Tom Brady', 'Gisele Bundchen', 'Eric Grimson']
2  Sorted by first name = ['Eric Grimson', 'Gisele Bundchen', 'Tom Brady']
```

Ejercicio manual

Usa `merge_sort` para ordenar una lista a tuplas de enteros. El orden de ordenamiento debe ser determinado por la suma de los enteros en la tupla. Por ejemplo, (5, 2) debe preceder a (1, 3) y seguir a (1, 2, 3).

```
1  def tupleInt(num1, num2):
2      arg1 = sum(num1)
3      arg2 = sum(num2)
4      if arg1 != arg2:
5          return arg1 < arg2
6      else:
7          return len(num1) < len(num2)
```

Ordenamiento en Python

El algoritmo de ordenamiento usado en la mayoría de las implementaciones de Python se llama **timsort**. La idea clave es aprovechar el hecho de que en muchos conjuntos de datos, los datos ya

están parcialmente ordenados. El rendimiento del peor caso de Timsort es el mismo que el de merge sort, pero en promedio se desempeña considerablemente mejor.

Como se mencionó anteriormente, el método de Python `list.sort` toma una lista como su primer argumento y modifica esa lista. En contraste, la función `sorted` toma un objeto iterable (por ejemplo, una lista o una vista) como su primer argumento y devuelve una nueva lista ordenada. Por ejemplo, el código

```
1  L = [3,5,2]
2  D = {'a':12, 'c':5, 'b':'dog'}
3
4  print(sorted(L))
5  print(L)
6  L.sort()
7  print(L)
8  print(sorted(D))
9  D.sort()
```

Imprimira

```
1  [2, 3, 5]
2  [3, 5, 2]
3  [2, 3, 5]
4  ['a', 'b', 'c']
5  AttributeError: 'dict' object has no attribute 'sort'
```

Observa que cuando la función `sorted` se aplica a un diccionario, devuelve una lista ordenada de las claves del diccionario. En contraste, cuando el método `sort` se aplica a un diccionario, causa una excepción porque es levantada ya que no existe tal método.

Tanto el método `list.sort` como la función `sorted` pueden tener dos parámetros adicionales. El parámetro `key` cumple el mismo papel que `compare` en nuestra implementación de merge sort: proporciona la función de comparación a ser usada. El parámetro `reverse` especifica si la lista debe ser ordenada en orden ascendente o descendente relativo a la función de comparación. Por ejemplo, el código

```
1  L = [[1,2,3], (3,2,1,0), 'abc']
2  print(sorted(L, key = len, reverse = True))
```

ordena los elementos de L en orden inverso de longitud e imprime

```
1  [(3, 2, 1, 0), [1, 2, 3], 'abc']
```

Tanto el método `list.sort` como la función `sorted` proporcionan **ordenamientos estables**. Esto significa que si dos elementos son iguales con respecto a la comparación (`len` en este ejemplo) usada en el ordenamiento, su ordenamiento relativo en la lista original (u otro objeto iterable) se preserva en la lista final. (Dado que ninguna clave puede ocurrir más de una vez en un `dict`, la cuestión de si `sorted` es estable cuando se aplica a un `dict` es discutible.)

Ejercicio manual

¿Es `merge_sort` un ordenamiento estable?

Nuestra implementación de `merge_sort` no es estable ya que `compare(left[i], right[j])` devuelve `False`, elige `right[j]`, **incluso si ambos elementos son "iguales"** según `compare`. Esto rompe la estabilidad, porque pone primero a `right[j]`, aunque `left[i]` haya aparecido antes en la lista original.